RECONNECT — Documentação técnica

Portal do **Retiro de Jovens RECONNECT** (05 a 09/02/2027): informações do evento, inscrição de vários participantes, pagamento, consulta pública e painel administrativo.

> Este documento descreve o sistema como ele está no repositório. **Não coloque senhas, chaves ou dados de inscritos aqui.** Os segredos ficam só nas variáveis de ambiente do Netlify.

---

1. Visão geral

Camada	Tecnologia	Onde roda
Site e painel	HTML, CSS e JavaScript puro (sem framework, sem build)	Netlify (arquivos estáticos em `public/`)
API	Netlify Functions (Node 18+, sem dependências, usa `fetch`)	Netlify (`netlify/functions/`)
Banco de dados	Supabase (PostgreSQL), acessado por REST (PostgREST)	Supabase
Login do painel	Supabase Auth (e-mail + senha)	Supabase
Pagamento no cartão (opcional)	Checkout hospedado da InfinitePay	InfinitePay
Teste local	`server.js` (Node puro, grava em `data/registrations.json`)	Computador do desenvolvedor

Fluxo geral:

```
Navegador ──► Netlify (arquivos estáticos)
   │
   └─► /api/*  ──► Netlify Functions ──► Supabase (REST, chave de serviço)
                          │
                          ├─► Supabase Auth (login do painel)
                          └─► InfinitePay (link de pagamento + conferência)
```

Publicação: cada **push na branch** `main` do GitHub dispara um deploy automático no Netlify (≈ 30 s). Cada deploy consome créditos do plano do Netlify.

---

2. Estrutura do repositório

```
├── config.json                Regras de negócio e textos configuráveis
├── netlify.toml               Publicação, funções e redirecionamentos (/api/*)
├── package.json               Só define "npm start" (node server.js)
├── server.js                  Servidor local para testes (não vai para produção)
├── .env.example               Modelo das variáveis de ambiente
├── supabase/schema.sql        Criação da tabela e das regras de segurança
├── docs/DOCUMENTACAO-TECNICA.md   Este arquivo
├── netlify/functions/
│   ├── _shared.js             Utilitários: config, Supabase REST, login do painel, validações
│   ├── _infinitepay.js        Integração com o checkout da InfinitePay
│   ├── register.js            POST /api/register   (cria a inscrição)
│   ├── lookup.js              POST /api/lookup     (consulta pública)
│   ├── config.js              GET  /api/config     (config pública)
│   ├── vagas.js               GET  /api/vagas      (contador de vagas)
│   ├── pay.js                 POST /api/pay        (link de pagamento no cartão)
│   ├── infinitepay-webhook.js POST /api/infinitepay-webhook
│   └── admin.js               /api/admin/*         (painel)
└── public/
    ├── index.html             Site principal
    ├── consulta.html          "Minha inscrição" (consulta por protocolo ou CPF)
    ├── admin.html             Painel administrativo (uma página só)
    ├── autorizacao.html       Modelo imprimível de autorização de menores
    ├── css/style.css          Estilos do site
    ├── js/content.js          Conteúdo editável: programação, local, novidades, fotos
    ├── js/main.js             Comportamento do site (menu, carrossel, contagem, galerias)
    ├── js/form.js             Formulário de inscrição, pagamento, aviso de vagas
    ├── js/pricing.js          Regras de preço e pagamento (compartilhado)
    └── img/                   Logos, fotos do local, line-up, etc.
```

`public/js/pricing.js` é carregado **pelo navegador, pelo** `server.js` **e pelas funções do Netlify** (via `included_files` no `netlify.toml`). Isso garante que o valor mostrado ao usuário é o mesmo que o servidor valida e grava.

---

3. Configuração (`config.json`)

Arquivo único com as regras. Para alterar, edite e publique (push).

Chave	Função
`registrationOpen`	`true` abre as inscrições; `false` mostra "Aguarde". Pode ser forçada pela variável `REGISTRATION_OPEN`.
`eventName`, `theme`, `eventStart`, `eventEnd`	Dados do evento (a contagem regressiva usa `eventStart`).
`prices`	Preço cheio por acomodação: `camping`, `dormitorio`.
`discountPercent`	Desconto em porcentagem (hoje 10) aplicado **sobre o valor total** da inscrição, no Pix à vista e no combo familiar.
`combo.minParticipants`	Mínimo de participantes para o combo familiar (hoje 3).
`installmentFees`	Taxa (%) do cartão por número de parcelas (1 a 12), conforme a InfinitePay.
`cardInstallments`	Parcelas liberadas no cartão (hoje 2 a 12).

Chave	Função
`maxInstallments`	Máximo de parcelas no cartão (12).
`cardDistricts`	Distritos que podem parcelar no cartão (hoje Veiga Jardim e Garavelo).
`crediario`	`{ enabled, maxInstallments, until }`: parcelamento mensal controlado pelo painel (`until` = data limite; `null` = sem limite).
`districts`	Opções de "Convidado por".
`contacts`	Responsável de cada distrito: nome, WhatsApp, chave Pix e titular. Usado no pagamento e no rodapé "Dúvidas".
`capacity`	`{ total, warnAt }`: limite de vagas (250) e quando mostrar o aviso de últimas vagas (20).
`pix`	Reservado (`key`, `holder`); hoje o Pix usa as chaves de cada distrito em `contacts`.

O texto da programação, local, novidades e fotos fica em `public/js/content.js`.

---

4. Regras de negócio

4.1 Preços (`public/js/pricing.js`)

- **Faixa etária:** 0–4 anos grátis · 5–9 anos meia inscrição (50%) · 10+ integral.
- **Pix (à vista):** desconto de 10% sobre o valor total (`discountPercent`). Ex.: camping R$ 220 → R$ 198; dormitório R$ 240 → R$ 216; camping + dormitório R$ 460 → R$ 414.
- **Crediário:** 2x ou 3x, uma parcela por mês, sem acréscimo.
- **Cartão:** valor da inscrição (com os 10% se for combo familiar) ÷ (1 − taxa). A taxa da maquininha é repassada ao cliente, e a organização recebe o preço-base. Exemplo (R$ 220): 3x = R$ 236,59; 12x = R$ 263,98.
- **Combo familiar:** com 3 ou mais participantes e a opção marcada, vale o preço com desconto também no parcelado. O desconto não acumula com o do Pix.
- É preciso ao menos um participante com 5 anos ou mais (quem paga).
- O primeiro participante é o responsável: o "Convidado por" dele define o distrito para pagamento.

4.2 Status da inscrição

`pre-cadastro` → `confirmada` (após o pagamento) → ou `cancelada`.

- No crediário, pagar a **1ª parcela** confirma a inscrição; todas pagas = quitado (`syncPlan`).
- No cartão pelo link da InfinitePay, o aviso de pagamento confirma sozinho.

4.3 Vagas

- Limite em `capacity.total` (250). **Só inscrições** `confirmada` **ocupam vaga**; cada participante conta 1.
- `GET /api/vagas` devolve `{ total, taken, remaining, warnAt }`.
- Quando `remaining ≤ warnAt` o site mostra um aviso chamativo (quadro e botão flutuante). Quando chega a 0: "Vagas esgotadas", com os contatos dos responsáveis, e o formulário some. O servidor também recusa novas inscrições (`409`).

- Pré-cadastros não reservam vaga; quem confirma depois do limite precisa ser tratado no painel.

4.4 Dados coletados

- **Todos os participantes:** nome completo, idade, CPF (opcional para menores de 18 com "sem CPF"), telefone, "Convidado por", acomodação, e quatro perguntas de saúde (alergia, restrição alimentar, medicamento controlado, acessibilidade).
- **Só o responsável:** contato de emergência (nome, telefone, parentesco) e **autorização de uso de imagem (obrigatória)**.
- **Menores de idade:** se não houver responsável legal, é exigida a ciência da **autorização assinada**, entregue no evento (modelo em `/autorizacao.html`).
- A consulta pública **nunca** mostra dados de saúde nem contato de emergência.

---

5. Banco de dados (Supabase)

Tabela única `public.registrations` (script em `supabase/schema.sql`):

Coluna	Tipo	Conteúdo
`id`	uuid	Chave primária
`protocol`	text, único	`RC-` + 6 caracteres hexadecimais (ex.: `RC-A1B2C3`)
`status`	text	`pre-cadastro`, `confirmada` ou `cancelada`
`created_at`	timestamptz	Data do cadastro
`participants`	jsonb	Lista de participantes (nome, idade, cpf, phone, district, lodging, health)
`cpfs`	text[]	CPFs para busca e bloqueio de duplicidade (índice GIN)
`payment`	jsonb	Forma de pagamento e valores (ver abaixo)
`info`	jsonb	Contato de emergência, uso de imagem, menores de idade

Campos de `payment`: `method` (`pix` / `crediario` / `cartao`), `installments`, `feePercent`, `subtotal`, `discount`, `combo`, `total`, `paid`, e conforme o caso `plan` (parcelas do crediário), `checkoutUrl` (link da InfinitePay), `paidAt`, `gateway`, `transactionNsu`, `paidAmount`, `paidInstallments`, `receiptUrl`.

Segurança: a tabela tem RLS ligada e **nenhuma policy**, e o acesso de `anon` e `authenticated` foi revogado. O navegador não lê nem grava nada diretamente. Só as funções do Netlify, com a chave de serviço, acessam os dados.

---

6. API

Todas as rotas `/api/*` são redirecionadas pelo `netlify.toml` para as funções.

Públicas

Rota	Método	Função
`/api/config`	GET	Configuração pública (sem a chave Pix global). Cache de 60 s.
`/api/vagas`	GET	Contador de vagas. Cache de 20 s.

Rota	Método	Função
`/api/register`	POST	Valida e grava a inscrição. Devolve protocolo, total e (cartão) `payUrl`.
`/api/lookup`	POST	Consulta por protocolo (`RC-XXXXXX`) ou CPF. Devolve nomes abreviados e CPF mascarado; espera de 300 ms por consulta.
`/api/pay`	POST	Devolve o link de pagamento no cartão de uma inscrição (por protocolo).
`/api/infinitepay-webhook`	POST	Aviso de pagamento aprovado da InfinitePay.

`/api/register` valida no servidor: de 1 a 15 participantes, nome completo, CPF válido e sem repetição, telefone, "Convidado por" permitido, acomodação, saúde, contato de emergência, imagem, menores, forma de pagamento, limite de vagas e CPF já inscrito (ignora canceladas).

Painel (`/api/admin/*`)

Rota	Método	Função
`login`	POST	E-mail + senha. Não exige estar logado.
`refresh`	POST	Renova a sessão com o `refresh_token`.
`data`	GET	Lista todas as inscrições.
`status`	POST	Confirma, cancela ou desfaz uma inscrição.
`installment`	POST	Marca ou desmarca uma parcela do crediário.
`export.csv`	GET	Exporta o CSV (inclui saúde, emergência e valor líquido).
`users`	GET / POST	Lista e cria usuários do painel.
`user-delete`	POST	Remove um usuário criado pelo painel.

Todas, exceto `login` e `refresh`, exigem o cabeçalho `Authorization: Bearer <token>`.

---

7. Autenticação do painel

- **Login:** a tela `/admin` pede e-mail e senha. A função `admin.js` chama o Supabase Auth (`/auth/v1/token`) e devolve a sessão (`access_token` , `refresh_token` , `expires_at`).
- **Verificação:** a cada chamada, o token é validado em `/auth/v1/user`. O usuário só entra se (a) o e-mail estiver em `ADMIN_EMAILS` ou (b) tiver `app_metadata.role = "admin"` (contas criadas pela tela **Usuários**).
- **Sessão:** guardada em `sessionStorage` do navegador; renovada automaticamente quando faltam menos de 90 s para expirar.
- **Proteções:** espera de 600 ms em login errado; mensagens genéricas ("E-mail ou senha incorretos").
- **Criar usuários:** a tela **Usuários** usa a API de administração do Supabase Auth com a chave de serviço (e-mail confirmado automaticamente e `app_metadata.role = "admin"`). O cadastro público no Supabase deve ficar **desligado**.
- **Modo temporário:** se `SUPABASE_PUBLISHABLE_KEY` não existir, vale uma senha única (`ADMIN_PASSWORD` , enviada em `x-admin-password`). Não deve ser usado em produção.

---

8. Pagamentos

8.1 Fluxo manual (sempre disponível)

Após o pré-cadastro, a pessoa vê os contatos e as chaves Pix de cada distrito, com a orientação de **conferir o nome do titular** e **enviar o comprovante ao responsável do distrito**. A mensagem de WhatsApp já vai com protocolo, número de vagas e valor total. O responsável confirma o pagamento no painel (ou marca as parcelas do crediário). No cartão, ele pode gerar o link na própria InfinitePay.

8.2 Link automático da InfinitePay (cartão)

Ativado quando existe a variável `INFINITEPAY_HANDLE` (InfiniteTag, sem `$`). Sem ela, tudo segue pelo fluxo manual.

1. `register.js`, para inscrições no cartão, chama `ensureLink` e devolve `payUrl`. O botão **Finalizar pagamento** aparece na confirmação e na consulta (`/consulta`).
2. `POST https://api.checkout.infinitepay.io/links` com `handle`, `order_nsu = protocolo`, item único com o **preço-base** em centavos, `redirect_url` (`/consulta?protocolo=...`) e `webhook_url` (`/api/infinitepay-webhook`). A InfinitePay soma a taxa do parcelamento conforme as parcelas escolhidas no checkout.
3. O link fica em `payment.checkoutUrl`.
4. Ao pagar, a InfinitePay envia o aviso ao webhook. **O aviso não é assinado**, então o servidor: (a) exige que o `order_nsu` seja um protocolo existente; (b) confirma o pagamento em `POST /payment_check`; (c) compara o valor da fatura com o preço-base; (d) só então grava `paid = true` e `status = confirmada`. Reenvios do mesmo `transaction_nsu` são ignorados.
5. Falhas do webhook retornam `400` (a InfinitePay tenta de novo). Pedido desconhecido ou valor diferente retornam `200` sem alterar nada.

---

9. Variáveis de ambiente (Netlify)

Variável	Obrigatória	Função
`SUPABASE_URL`	Sim	URL do projeto Supabase (aceita com ou sem `/rest/v1`).
`SUPABASE_SERVICE_ROLE_KEY`	Sim	Chave **secreta** do Supabase (`sb_secret_...` ou `service_role`). Nunca a publishable.
`SUPABASE_PUBLISHABLE_KEY`	Para o login por e-mail	Chave pública do Supabase (usada no Supabase Auth).
`ADMIN_EMAILS`	Recomendado	E-mails com acesso fixo ao painel, separados por vírgula. Podem ser os primeiros administradores.
`INFINITEPAY_HANDLE`	Para o link automático	InfiniteTag sem `$`.
`REGISTRATION_OPEN`	Não	`true` / `false` para abrir ou fechar as inscrições sem editar `config.json`.
`ADMIN_PASSWORD`	Não (modo temporário)	Só vale sem `SUPABASE_PUBLISHABLE_KEY`.

Depois de criar ou alterar uma variável, faça um novo deploy (**Deploys** → **Trigger deploy**).

---

10. Segurança e LGPD

- Segredos só no Netlify; o repositório pode ser público (não há segredo nem dado pessoal nele).
- Banco sem acesso direto do navegador (RLS + revogação); toda leitura e escrita passa pelas funções.
- Consulta pública mostra apenas nomes abreviados, CPF mascarado e dados de pagamento; nunca saúde nem emergência.
- Entradas são validadas no servidor (CPF, telefone, valores recalculados com `pricing.js`); o valor enviado pelo navegador nunca é confiado.
- Painel: sessão por token verificado no Supabase, lista de acesso (`ADMIN_EMAILS` / `role=admin`), cabeçalho `noindex` e `no-store` em `/admin`.
- O painel permite apagar usuários criados por ele; e-mails de `ADMIN_EMAILS` só saem pelo Netlify.
- O banco guarda CPF, telefone e dados de saúde: limite quem tem acesso ao Supabase e ao Netlify, e **apague os dados depois do evento**.
- Cuidado com a chave Pix pública e com o link de pagamento: quem souber o protocolo consegue abrir o link de pagamento da inscrição (só permite pagar, não ver dados).

---

11. Operação

Abrir ou fechar inscrições

`config.json` → `"registrationOpen": true|false` e push, ou variável `REGISTRATION_OPEN` + novo deploy.

Alterar valores e taxas

Edite `prices`, `discountPercent` e `installmentFees` em `config.json`. As taxas são calculadas por dentro (preço ÷ (1 − taxa)); a cada nova tabela da InfinitePay, recalcule a taxa de cada parcela a partir dos valores de R$ 220.

Usuários do painel

Pelo botão **Usuários** do painel (ou em Supabase → Authentication → Users). Mantenha o cadastro público do Supabase desligado.

Limpar inscrições de teste

No Supabase (SQL Editor): `delete from registrations;`. Irreversível; baixe o CSV antes se houver dados reais.

Fotos

Edite `public/js/content.js` (`photos` para os 6 destaques e `photosLink` para o álbum) e coloque as imagens em `public/img/retiro/`.

Depois do evento

Exportar o CSV, apagar os dados pessoais do banco e, se desejado, fechar as inscrições.

---

12. Desenvolvimento local

```
node server.js      # http://localhost:3000 (admin em /admin)
```

- Dados em `data/registrations.json` (ignorado pelo Git); apague o arquivo para limpar.
- Login do painel local: qualquer e-mail + `ADMIN_PASSWORD` (padrão `troque-esta-senha`).
- Porta: `PORT`. Abertura: `REGISTRATION_OPEN`.
- **Não funcionam localmente:** link automático da InfinitePay, webhook e a tela **Usuários** (dependem do Netlify e do Supabase).
- Depois de alterar `server.js` ou `pricing.js`, reinicie o servidor.

Deploy

`git push` na `main` → Netlify publica sozinho. Confira o resultado com `curl` ou no navegador (Ctrl+F5 para limpar o cache). Se o plano do Netlify estiver sem créditos, o push chega ao GitHub mas **o site não é publicado**.

---

13. Limitações conhecidas e ideias futuras

- Pré-cadastros não reservam vaga; o total confirmado pode passar de 250 se várias confirmações forem feitas ao mesmo tempo (a trava só vale para novos cadastros).
- A resposta de criação do link da InfinitePay não é documentada em detalhe; o código procura a primeira URL `https` na resposta.
- O aviso (webhook) da InfinitePay não é assinado; a proteção é a conferência em `payment_check`.
- O Pix pelas chaves dos distritos é confirmado manualmente. Um link automático de Pix pelo mesmo checkout é possível, mas ainda não foi implementado.
- Controle de vagas por quarto ainda não existe (aguardando a capacidade de cada quarto).
- Consolidar vários eventos no mesmo painel é uma ideia futura (nada implementado).